

Individual assignment cover sheet

Course Name:	Master of Information Systems
Unit Number and Name:	MIS5303 Securing Software
Lecturer Name:	Dr Omid Haass
Assignment No.:	02

Declaration:

By submitting this declaration, I, the below-named student, declare the following:

- ✓ Except where appropriately acknowledged, this assignment is my own work and has been expressed in my own words.
- ✓ Where I have made use of generative AI software, I have done so appropriately and cited it accurately as per the '**HE-AP19 Generative Artificial Intelligence (AI) Guidelines for Students**' document found in the LMS, See: [HE-AP19 Generative Artificial Intelligence \(AI\) Guidelines for Students.pdf \(apexhe.nsw.edu.au\)](https://apexhe.nsw.edu.au/policiesandprocedures/).
- ✓ This assignment in its entirety, or this particular part of it, has not previously been submitted by me or any other person for assessment on this or any other course of study.
- ✓ I have read the Student Academic Misconduct Policy and Procedure found at <https://apexhe.nsw.edu.au/policiesandprocedures/>, and I understand the significance of academic integrity and the consequences of breaching it.

Use of Generative Artificial Intelligence (AI)

Please tick one of the following options:

- ✓ **Declaration 1:** I acknowledge using AI tool(s) to generate content that are included in this work in modified forms.
- ✓ **Declaration 2:** I acknowledge the use of AI tool(s) in the preparation of material that aids the process of completion of this assignment (e.g. for self-study or background reading). No materials are submitted as original work on our part.
- ✓ **Declaration 3:** I did not use any content generated by AI tools in the preparation of this assignment

If your response was **Declaration 1** or **Declaration 2**, please complete the following:

	Student's Response
AI Tool(s) Used (e.g., ChatGPT, Grammarly, DALL-E 2, Pi Assistant, Lumen5, Looka, etc.)	ChatGPT
Purpose of Use (Explain specifically what the tool was used for – e.g., idea generation, grammar correction, image generation, summarizing content, etc.)	Idea Generation
List all prompts you have used	
Extent of Modification (Explain if and how you rewrote or modified the AI-generated content. E.g.,	Took the idea and explain in my own words.

rephrased, fact-checked, rearranged, quoted suitably, etc.	
Consider the manners in which your work is different from AI-generated contents.	
How have you modified the generative AI's output?	

Note: All AI usage must be acknowledged and comply with the HE-AP19 Generative Artificial Intelligence (AI) Guidelines for Students. Unacknowledged use of AI tools constitutes academic misconduct.

Student ID Number:	Student Name:
240135	Ashraful Islam

Signed:

Date: 08-Aug-2026



Recoverable Signature

X Ashraful Islam

Name

Signed by: 341b0587-a755-4806-a88b-56fe95828d8f

1. Introduction

There are always new pieces of software hitting the market and if they are not considered during this audit, there is a possibility that there is some hole through which an organisation can get breached and see money leak out and their reputation soiled. Need to be able to discover these vulnerabilities before, and as software runs on applications. This report analyses, analyses and compares secure software development tools: Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), Interactive Application Security Testing (IAST) and Runtime Application Self-Protection (RASP) in four major categories. Instead of vendor documentation, a representative, real-world tool is analyzed in each category for its working mechanism, strengths and weaknesses, usability, and evidence that is gleaned from independent research. These secure development practices are summarized in a set of practices at the end of the report that are based on this evidence and aligned with well-known assurance frameworks.

2. Static Application Security Testing (SAST)

But SAST tools do not execute the code or application binary instead, they conduct a scan of security coding patterns including those that might be applied in injecting attacks, unsafe code deserialization, and hard coded secrets (Li, 2020). An example of this is the popular bandit, an open-source SAST tool that web crawls an AST of python code, and applies a pattern on the code by using a set of rules - which is viable enough to be run on each commit.

The big advantage of SAST is the early detection, as it runs before you merge or deploy code and is the least expensive time block in the SDLC to fix a defect and doesn't require the running version of the application. Another objective perspective to benchmarking on its own though is a sober one- a sober perspective of benchmarking is the vendors will not. In this regard, Li et al. (2023) evaluated seven of the most popular SAST tools and found that the tools together can only identify 12.7% of the known vulnerabilities, most of which are not present in most tools and most vulnerabilities needed execution context or neutralize input across the different trusted domains. Similarly, SAST is vulnerable to a large number of false positives since it cannot make conjectures about the state of runtime which causes developers to lose confidence. The overall usage is reasonably good: Open source SAST tools easily integrate with pre-commit hooks or CI pipelines, and have active communities; enterprise versions are expensive.

3. Dynamic Application Security Testing (DAST)

DAST tools operate on the running app as a "black box" that attempts to craft requests and watch for responses to identify possible runtime vulnerabilities, such as cross-site scripting, injection, and broken authentication (Somi, 2024). The primary open-source alternative is OWASP ZAP that performs proxying, active spiders, files and logs the vulnerabilities in form of steps to replicate them.

The primary benefit of DAST is that it is based on observed behaviour, and there is a lower ratio of false-positives than with SAST, and no need to access the source code, so it's suitable for third-party or legacy assessment projects. Its most revealing weakness is that it is unaware of the internal architectures i.e. does not know what code path emitted a reply and it can barely stubbornly follow an authenticated workflow or a nested workflow to the methods without additional configuration. The goals of NSEwithee Somi (2024) were to establish a benchmark of the most popular DAST tools, and identify the difference in the coverage and false-negative rates of the products depending on how each journalises through user-friendly front ends. DAST isn't as fast, and is not run as frequently as is viable – most teams run on staging instead of on every commit. ZAP does not cost anything, is scriptable, yet it still needs to be adjusted to disregard the authentication and session management, which is an excellent constraint of effective security competence.

4. Interactive Application Security Testing (IAST)

The IAST instruments offer code-level visibility and evidence acquired by the runtime code of a system during the system-level functional tests, thus the two sources (code and runtime) can exist side by side under the same name, IAST (Seth et al., 2025). A typical commercial one is one that compares contrast security wherein there is an agent embedded into the application server.

The findings of vulnerability detection by IAST versus SAST as used by other authors on a large production Java system on the same platform [3] revealed that IAST is second most effective in terms of money spent to vulnerabilities detected, compared to both manual (expert) penetration into the system and alone SAST gives a significantly lower false-positive rates. IAST is constructed atop legacy functional tests suites meaning only a low level of effort is needed to tool them once they have been instrumented and they will give a true line location and a live proving that they are being exploited. The primary drawback of this, sadly, is the so-called coverage dependence that a failure to run underlying tests will leave some checks unexercised, and therefore will not lead to the security coverage or in the case of instrumentation with agents, there will be a quantifiable performance cost (Seth et al., 2025). There are more cost and efforts required in implementation of available IAST products compared to open source SAST and DAST solutions.

5. Runtime Application Self-Protection (RASP)

RASP exists on the other side of the pipeline, testing the application after it is live to reveal vulnerabilities, but also adding a protective layer inside the production application to catch and prevent exploitation attempts as they happen—hence the term, a last line of defence (Fry, 2019). Contrast Protect is a representative commercial RASP agent which observes application behaviour and takes action when input matches with input payload against application attack signatures.

In production RASP achieves very low false-positive rates against DAST and IAST, and in the evaluation carried out by SANS for RASP, the performance was shown to be effective at blocking known exploitation attempts against vulnerable target applications, with very low false positives – especially among well-understood attack classes (Fry, 2019). RASP's disadvantage is that it is not a discovery tool, sufficient to decrease the count of vulnerabilities, but rather is a compensating control because it cannot be attacked if the pattern was not modelled by RASP, and because each application will likely have to be tuned to it before it can be deployed (Fry 2019). Cost and vendor lock-in are also salient because most of the well-established RASP products are proprietary.

6. Comparative Evaluation

Table 1 summarises the four categories against the criteria used in the appraisal above.

Tool/Technique	Example Tool	Type	Strengths	Weaknesses	Integration	Notable Use Cases
SAST	Bandit	SAST	Fast; runs pre-commit/CI; identifies vulnerable code patterns prior to merge; requires no runtime state.	Detects only ~12.7% of real-world vulnerabilities in benchmarks; high false-positive rate; blind to runtime state (Li et al., 2023)	Easy – pre-commit hook, CI gate	Python codebases; PR/merge gating
DAST	OWASP ZAP	DAST	Identifies exploitable runtime issues; does not require access to source code; has low false positives on known exploits	Is Blind to code path/logic; has slow resource-intensive scans (Somi, 2024)	Moderate – needs auth/session tuning	Web app pentests; staging scans
IAST	Contrast Security	IAST	Second-most efficient technique after manual pentest; exact code location; low false positives (Seth et al., 2025)	Coverage limited to what QA/functional tests exercise; runtime performance overhead	Moderate – agent instrumentation	Apps with mature automated test suites
RASP	Contrast Protect	RASP	Blocks known attack patterns on production in real time; false positives on known attacks are low (Fry, 2019)	Compensating control only - does not fix root cause; can be bypassed with new patterns; requires per-application tuning	Harder – embedded in production runtime	Internet-facing production systems

7. Secure Development Practices and Recommendations

On this evidence and consistent programme, there are three recommendations.

Pre-merge, on every commit, run SAST. OWASP suggests that as a part of its Secure Build practice, teams should use at least two different SAST engines that complement each other so that SAST is used as a quick first “defense” in the pipeline and not as a full assurance: “Automated checks should be included in the build pipeline, using at least two complementary SAST engines (as found by Li et al., 2023) and treating SAST as a fast screening step rather than an assurance” (OWASP, 2020).

In case of feasible test suites, run DAST or IAST indefinitely on all staging deployments. The entire runtime and configuration space, which DAST has closed, is the one SAST cannot observe, and the discovery of IAST have been found to be both on the order of low false positive, code located and are excellent in direct developer triage without a dedicated security team- discovered by the DevSecOps maturity research (Fucci et al., 2024) has identified a diluted-ownership risk.

RASP should be used as a counter balanced control with systems at the edge of the internet in an application production setting, and not to replace the above. It comes into accordance with the recommendations in the Essential Eight section of the Australian Signals Directorate (2023) which requires the use of layered mitigation strategies in combination and aligns with the recommendation in the ISO/IEC 27001 of controls of a secure system development lifecycle (SDLC) (ISO/IEC, 2022).

The three are all underpinned using OWASP SAMM to periodically measure a practice an organisation may have and then measure against the perception. As discussed by Fucci et al. (2024), large gaps in perception and reality of security maturity were identified, underpinning all three using OWASP SAMM to periodically measure the security practices an organisation has and compares to the perception.

8. Conclusion

None of these types of tools are complete on their own. While the real world is load dependent, and IAST and RASP both have trade-offs on structural coverage, RASP is best at addressing what SAST and DAST are unable to handle efficiently, while SAST is both cheap and early but structurally blind, DAST is expensive and slow but validates exploitability, and IAST will have the best efficiency-to-accuracy ratio when appropriate and sufficient structural coverage is maintained. The evidence reviewed here confirms a layered approach of SAST, DAST/IAST, and RASP pipeline embedded in CI/CD, and regularly compared to a maturity model, like OWASP SAMM.

References

- Australian Signals Directorate. (2023). Essential Eight maturity model. Australian Cyber Security Centre. <https://www.cyber.gov.au/resources-business-and-government/essential-cyber-security/essential-eight/essential-eight-maturity-model>
- Fry, A. (2019). Runtime Application Self-Protection (RASP): Investigation of the effectiveness of a RASP solution in protecting known vulnerable target applications. SANS Institute. <https://www.sans.org/white-papers/38950>
- Fucci, D., Alégroth, E., Felderer, M., & Johannesson, C. (2024). Evaluating software security maturity using OWASP SAMM: Different approaches and stakeholders' perceptions. *Journal of Systems and Software*, 214, Article 112062. <https://doi.org/10.1016/j.jss.2024.112062>
- International Organization for Standardization/International Electrotechnical Commission. (2022). ISO/IEC 27001:2022 – Information security, cybersecurity and privacy protection – Information security management systems – Requirements. ISO.
- Li, J. (2020). Vulnerabilities mapping based on OWASP-SANS: A survey for static application security testing (SAST). arXiv. <https://doi.org/10.48550/arXiv.2004.03216>
- Li, K., Chen, S., Fan, L., Feng, R., Liu, H., Liu, C., Liu, Y., & Chen, Y. (2023). Comparison and evaluation on static application security testing (SAST) tools for Java. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023)* (pp. 921–933). Association for Computing Machinery. <https://doi.org/10.1145/3611643.3616262>
- Open Web Application Security Project. (2020). OWASP Software Assurance Maturity Model (SAMM) v2.0. <https://owasp samm.org/>
- Seth, A., Bhattacharya, S., Elder, S., Zahan, N., & Williams, L. (2025). Comparing effectiveness and efficiency of Interactive Application Security Testing (IAST) and Runtime Application Self-Protection (RASP) tools in a large Java-based system. *Empirical Software Engineering*, 30, Article 92. <https://doi.org/10.1007/s10664-025-10621-5>
- Sharma, M. (2021). Review of the benefits of DAST (dynamic application security testing) versus SAST. *International Journal of Management and Engineering Research*, 1(1), 5–8.
- Somi, V. (2024). A comparative analysis and benchmarking of Dynamic Application Security Testing (DAST) tools. *Journal of Engineering and Applied Sciences Technology*, 6(6). [https://doi.org/10.47363/JEAST/2024\(6\)E139](https://doi.org/10.47363/JEAST/2024(6)E139)